

Python Interview Cheat Sheet (Easy to Remember)

For Quick Revision | Simple Examples | Core Concepts

1. Python Basics (Quick Recap)

Key Points

- Interpreted, dynamically typed, OOP, high-level language
- Indentation (4 spaces) = code blocks
- Everything is an **object**

Simple Example

代码块

```
1  # Variable (no type declaration)
2  name = "Python"
3  num = 10
4
5  # Function
6  def add(a, b):
7      return a + b
8
9  print(add(2,3))  # Output: 5
```

2. Decorators (Function + Class)

What is a Decorator?

- A function that **adds extra behavior** to another function/class **without modifying original code**
 - Uses `@` syntax (syntactic sugar)
-

A. Function Decorator

代码块

```
1  # Simple decorator: print before/after function call
2  def my_decorator(func):
3      def wrapper():
4          print("Before function runs")
5          func() # call original function
6          print("After function runs")
7      return wrapper
8
9  # Use decorator
10 @my_decorator
11 def greet():
12     print("Hello!")
13
14 greet()
```

Output:

代码块

```
1  Before function runs
2  Hello!
3  After function runs
```

Decorator with Arguments

代码块

```
1  def decorator(func):
2      def wrapper(*args, **kwargs): # accept any args
3          print("Start")
4          result = func(*args, **kwargs)
5          print("End")
6          return result
7      return wrapper
8
9  @decorator
10 def add(a,b):
11     return a+b
12
13 print(add(3,4)) # Output: 7
```

B. Class Decorator

Class acts as decorator → uses `__call__` method

代码块

```
1  class ClassDecorator:
2      def __init__(self, func):
3          self.func = func
4
5      def __call__(self, *args, **kwargs):
6          print("Class decorator running")
7          return self.func(*args, **kwargs)
8
9  @ClassDecorator
10 def say_hi():
11     print("Hi from function")
12
13 say_hi()
```

Output:

代码块

```
1  Class decorator running
2  Hi from function
```

3. Class Dunder (Magic) Methods

What are Dunders?

- Double underscore `__method__`
- Automatically called by Python (e.g., `__init__`, `__str__`)

Most Important Dunders (Easy to Remember)

代码块

```
1  class Person:
2      # 1. Constructor (called when object created)
3      def __init__(self, name, age):
4          self.name = name
```

```

5         self.age = age
6
7         # 2. String representation (print(object))
8         def __str__(self):
9             return f"Name: {self.name}, Age: {self.age}"
10
11        # 3. Length (len(object))
12        def __len__(self):
13            return self.age
14
15        # 4. Add objects (obj1 + obj2)
16        def __add__(self, other):
17            return self.age + other.age
18
19        # Usage
20        p1 = Person("Alice", 25)
21        p2 = Person("Bob", 30)
22
23        print(p1)           # __str__ → Name: Alice, Age: 25
24        print(len(p1))      # __len__ → 25
25        print(p1 + p2)     # __add__ → 55

```

Top 5 Interview Dunders

1. `__init__` → constructor
2. `__str__` → string output
3. `__len__` → length
4. `__add__` → addition
5. `__call__` → make object callable like function

4. Generator

What is a Generator?

- Function that **yields values one by one** (instead of returning all at once)
- Saves memory (lazy execution)
- Uses `yield` keyword

Simple Example

代码块

```

1  # Generator function
2  def count_up_to(n):
3      count = 1
4      while count <= n:
5          yield count    # pause & return value
6          count += 1
7
8  # Use generator
9  gen = count_up_to(3)
10
11 print(next(gen))    # 1
12 print(next(gen))    # 2
13 print(next(gen))    # 3

```

Generator Expression (One-Liner)

代码块

```

1  gen = (x for x in range(3))
2  print(list(gen))    # [0,1,2]

```

Key: Generators = memory efficient for large data

5. Context Manager

What is it?

- Automatically manages resources (open/close files, locks)
- Uses `with` statement
- Implemented with `__enter__` and `__exit__` OR `@contextmanager`

A. Class-Based Context Manager

代码块

```

1  class FileManager:
2      def __init__(self, filename, mode):
3          self.filename = filename
4          self.mode = mode
5
6      def __enter__(self):
7          self.file = open(self.filename, self.mode)
8          return self.file

```

```

9
10     def __exit__(self, exc_type, exc_val, exc_tb):
11         self.file.close()
12
13     # Usage (auto closes file)
14     with FileManager("test.txt", "w") as f:
15         f.write("Hello")

```

B. Function-Based (Decorator) → EASY

代码块

```

1  from contextlib import contextmanager
2
3  @contextmanager
4  def open_file(name):
5      f = open(name, 'w')
6      yield f # give control to block
7      f.close() # runs after block
8
9  # Usage
10 with open_file("test.txt") as f:
11     f.write("Python!")

```

Why use it? No need to manually `close()` → prevents resource leaks

6. asyncio (Asynchronous Programming)

What is asyncio?

- For **concurrent code** (no threads, single-threaded concurrency)
- Use `async` (define) + `await` (wait)

Simple Example

代码块

```

1  import asyncio
2
3  # Async function
4  async def task(name, delay):
5      print(f"Start {name}")
6      await asyncio.sleep(delay) # non-blocking wait

```

```
7     print(f"End {name}")
8
9     # Run multiple tasks
10    async def main():
11        # Run concurrently
12        await asyncio.gather(
13            task("A", 2),
14            task("B", 1)
15        )
16
17    # Run main
18    asyncio.run(main())
```

Output:

代码块

```
1   Start A
2   Start B
3   End B   # B finishes first (1s)
4   End A   # A finishes later (2s)
```

Key Terms

- `async def` → async function
- `await` → wait for async task
- `asyncio.run()` → run main function
- `asyncio.gather()` → run multiple tasks at once

Quick Interview Cheat List (Memorize This!)

1. **Decorator:** `@` → add behavior to func/class
2. **Dunder:** `__method__` → `__init__`, `__str__`, `__len__`, `__add__`
3. **Generator:** `yield` → lazy, memory efficient
4. **Context Manager:** `with` → auto resource clean-up
5. **asyncio:** `async` + `await` → single-thread concurrency

(注：文档部分内容可能由 AI 生成)